

hw13

PDF

can you please help me do the prelecture readings to tackle this homework assignment on diffusion. please start by outlining the core background concepts i need to know to tackle this homework

← I first prompted Gemini to give me the broad overview of all the concepts I would need to know, because that is generally how I study

◆ Show thinking ▾

Based on the homework file you uploaded, this assignment covers two distinct but cutting-edge areas of Deep Learning: **Generative Diffusion Models** (specifically the math behind DDPM/DDIM) and **Direct Preference Optimization (DPO)** (an alternative to RLHF).

Here is a breakdown of the core background concepts and "pre-lecture" mental models you need to tackle this assignment.

Part 1: Diffusion Models (DDPM & DDIM)

Relevance: Questions 1 and 4. These questions ask you to work through the probabilistic mechanics of diffusion, specifically looking at limits (as step size $\Delta t \rightarrow 0$) and the difference between stochastic (DDPM) and deterministic (DDIM) sampling.

1. The Forward Process (Diffusion)

- **Concept:** Diffusion models slowly destroy data structure by adding noise. The forward process $q(x_t|x_{t-1})$ adds a tiny amount of Gaussian noise at each step.
- **Key Property:** You can jump directly from the initial data x_0 to any noisy step x_t without iterating through all steps. This is done using the "reparameterization trick."
- **The Math:** If you add Gaussians, the result is Gaussian. The homework asks you to prove this property in Q4(a).
- Equation form: $q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$.

I wish it provided a less naive explanation with more examples instead of just breaking it down into the concept and math

2. The Reverse Process (Denoising)

- **Concept:** The goal of the model is to learn the *reverse* conditional distribution $p_\theta(x_{t-1}|x_t)$. In plain English: "Given this noisy image, what did it look like one step ago?"
- **Gaussian Assumption:** If the step size Δt is small enough, the reverse step is also approximately Gaussian. This is why we can train a neural network to simply predict the mean and variance of this Gaussian.

3. DDPM vs. DDIM

This is crucial for **Question 1 (f) and (g)**.

- **DDPM (Denoising Diffusion Probabilistic Models):** This process is **stochastic**. When you move backward from t to $t - 1$, you predict the clean image (or the noise), move towards it, and *add a little bit of fresh noise* back in. This random noise is essential for DDPM to generate diverse samples.
- **DDIM (Denoising Diffusion Implicit Models):** This process is **deterministic**. It assumes the same forward marginals as DDPM but defines a non-Markovian forward process where $\sigma \rightarrow 0$ for the reverse step.
- **Intuition:** DDIM views the diffusion process as an Ordinary Differential Equation (ODE). You can slide back and forth along the noise trajectory deterministically without adding new random noise.

it doesn't make sense why it arbitrarily iteratized this

4. The "Signal-to-Noise" Ratio

- **Homework Context:** Question 1 simplifies the problem by assuming the target is a simple Gaussian $X_0 \sim \mathcal{N}(0, \sigma^2)$.
- **Why?** This "toy problem" allows you to solve the diffusion equations analytically (with pen and paper) rather than training a neural net. It tests your ability to handle variances and limits. You will need to be comfortable with **Taylor expansions** (e.g., $\ln(1 - x) \approx -x$).

Part 2: RLHF & Direct Preference Optimization (DPO)

Relevance: Question 2. This question derives DPO, a method that aligns language models to human preferences without needing an explicit Reward Model or PPO (Proximal Policy Optimization).

1. The Standard RLHF Pipeline (The "Old" Way)

To understand DPO, you must understand what it replaces: 

1. **SFT (Supervised Fine-Tuning):** Train the model on high-quality data.
2. **Reward Modeling:** Train a separate neural network (the Reward Model) to predict which of two answers a human prefers.
3. **RL Fine-Tuning (PPO):** Use Reinforcement Learning to optimize the language model to maximize the score from the Reward Model.

I noticed that the explanations of key lecture points were very coarse and not helpful at the mathematical level of this class.

2. The Bradley-Terry Model

- **Concept:** A mathematical model for pairwise comparisons. It assumes the probability that answer y_w is preferred over y_l depends on the difference in their latent rewards. 
- **The Math:**

$$p(y_w > y_l | x) = \sigma(r^*(x, y_w) - r^*(x, y_l))$$

- where σ is the sigmoid function. You will use this in **Q2(e)**.

3. The Key Insight of DPO

- **Concept:** The optimization problem in RLHF (maximizing reward subject to a KL-divergence constraint) has a **closed-form solution**. 
- **The Derivation:**
 - Usually, we train a Reward Model $r(x, y)$ and then find the policy π .
 - DPO flips this algebra. It solves the optimal policy equation for the reward. This expresses the reward $r(x, y)$ in terms of the optimal policy π^* and the reference policy π_{ref} .
 - **Result:** You can plug this "implicit reward" directly into the Bradley-Terry model. This allows you to train the policy on preference data directly using a simple classification loss (Binary Cross Entropy), skipping the Reward Model and PPO entirely. 

I liked how it associated concepts

4. Plackett-Luce Model

- **Relevance:** Question 2(g).
- **Concept:** A generalization of Bradley-Terry for ranking more than two items (e.g., ranking 4 different summaries from best to worst). It calculates the probability of a specific permutation (ordering) of items. 

Part 3: Mathematical Toolkit Needed

To successfully complete the derivations in this homework, you should review:

1. **KL Divergence:** specifically the definition $\mathbb{D}_{KL}(p||q) = \sum p(x) \log \frac{p(x)}{q(x)}$. 
2. **Lagrange Multipliers:** You will likely need this intuition for Q2(b) to maximize the constrained objective.
3. **Gaussian Conditionals:** If (X, Y) are jointly Gaussian, what is $X|Y$? (Used heavily in Q1).
4. **Logarithm Rules:** Specifically $\log(a/b) = \log a - \log b$. This is used to convert the complex fraction inside the DPO derivation into a subtraction of log-probs. 

I then prompted it to go over this mathematical toolkit.

Recommended Next Step

Since **Question 1 (DDPM/DDIM Fun)** involves intricate variance calculations and limits, would you like me to walk you through the setup for **1(a) and 1(b)** to ensure you have the correct starting distributions for X_1 and $X_{t-\Delta t}$?

can you please help me ramp up on all 4 parts of the mathematical toolkit. please only cover what i would need to know for this homework assignment

This is a great strategy. Since the homework is split into "Diffusion" (Questions 1 & 4) and "DPO" (Question 2), these four tools are exactly what you need to unlock the derivations.

Here is your "Just-In-Time" toolkit, tailored specifically to what this assignment asks you to do.

Toolkit Part 1: Gaussian Conditionals (For Q1 & Q4)

You need this to handle the forward and backward diffusion processes.

1. Sum of Independent Gaussians

- **Concept:** If you have a value $X \sim \mathcal{N}(\mu_x, \sigma_x^2)$ and you add noise $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2)$, the result $Y = X + \epsilon$ is simply another Gaussian.
- **The Rule:** Means add; Variances add.
 - $Y \sim \mathcal{N}(\mu_x, \sigma_x^2 + \sigma_\epsilon^2)$.
- **Homework Application:** In **Q1(a)**, you start with X_0 (Gaussian) and add many small noise steps (Gaussians). This rule tells you immediately what the final distribution X_1 is.

2. Bayes' Rule for Gaussians

- **Concept:** The reverse diffusion step asks: "Given I'm at noisy step x_t , where did I likely come from (x_{t-1})?"
- **The Math:** $p(x_{t-1}|x_t) = \frac{p(x_t|x_{t-1})p(x_{t-1})}{p(x_t)}$.
- **The Shortcut:** When you multiply two Gaussian PDFs (the likelihood and the prior), the result is **always** a Gaussian (or proportional to one). You don't need to do complex integration. You just need to "complete the square" in the exponent to find the new mean and variance.
- **Homework Application:** **Q1(c)** gives you the formula for this conditional distribution. **Q4(b)** asks you to prove it.

Toolkit Part 2: Logarithms & Limits (For Q1)

You need this because Question 1 asks you to take the limit as the step size $\Delta t \rightarrow 0$ (turning sums into integrals).

1. The Taylor Approximation

- **The Rule:** When x is a tiny number (close to 0):
 - $\ln(1 - x) \approx -x$
 - $\ln(1 + x) \approx x$
- **Homework Application:** You will see expressions like $\ln(1 - \frac{\Delta t}{\dots})$. Use this rule to turn the complicated log into a simple subtraction. This is explicitly hinted in **Q1(h)**.

2. Products to Sums (Telescoping)

- **The Rule:** $\log(\prod a_i) = \sum \log(a_i)$.
- **Homework Application:** The total noise added over T steps is a product of many small variances (contractions). To analyze this as $\Delta t \rightarrow 0$, you take the log to turn that **product** into a **sum**, apply the Taylor approximation, and then turn that **sum** into an **integral**.
 - $\lim_{\Delta t \rightarrow 0} \sum f(t)\Delta t = \int f(t)dt$.

Toolkit Part 3: KL Divergence (For Q2)

You need this for the "Optimization" part of DPO.

1. The Definition

- **The Math:** For discrete probability distributions (like language models):

$$\mathbb{D}_{KL}(p||q) = \sum_x p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

*I wish it
dove into
the motivation more,
but maybe that would
require further
prompting.*

- **Intuition:** It measures "distance" between two distributions. It is always ≥ 0 , and is only 0 if $p = q$.
- **Homework Application:** In **Q2(a)** and **Q2(b)**, you are asked to minimize this value (or maximize a reward minus this value).

2. The "Log-Sum" Trick

- **The Math:** $\log \frac{a}{b} = \log a - \log b$.
- **Homework Application:** In **Q2(e)**, you will substitute a complex fraction into a log. Use this rule immediately to break it apart. You will find that the "partition function" $Z(x)$ appears in

both the numerator and denominator terms and cancels out perfectly.

Toolkit Part 4: Lagrange Multipliers (The "Hammer" for Q2)

This is the most critical tool for deriving the DPO solution in Q2(b).

The Problem: You want to find a probability distribution $\pi(y|x)$ that maximizes some Reward minus a KL penalty.

- **The Catch:** You can't just set the derivative to 0 because $\pi(y|x)$ must be a valid probability distribution (i.e., $\sum_y \pi(y|x) = 1$).

The Recipe: When optimizing probabilities with a sum-to-1 constraint:

1. **Write the Lagrangian:** $\mathcal{L} = \text{Objective} - \lambda(\sum_y \pi(y) - 1)$.
2. **Take the derivative** w.r.t. $\pi(y)$ and set to 0.
3. **Solve for $\pi(y)$.** You will usually get something resembling a "Softmax" or Boltzmann distribution: $\pi(y) \propto e^{\text{something}}$.
4. **Use the constraint** to find λ (or just call the normalization constant Z).

Homework Application: This is exactly how you solve Q2(b) to show the optimal policy has that exponential form involving the reward $r(x, y)$.

How to Start?

I suggest we tackle **Question 1(a)** first. It uses the "Sum of Independent Gaussians" tool.

Question 1(a): If we start at $X_0 \sim \mathcal{N}(0, \sigma^2)$ and add noise with variance Δt repeatedly for T steps (where $T \cdot \Delta t = 1$), what is the final variance of X_1 ?

(Give this a quick thought—do the variances just add up?)

can you please explain some key misconceptions faced by students?

◆ Show thinking ▼

This is a very smart way to prepare. These topics are conceptually dense, and students often trip up on specific "invisible" details.

Here are the key misconceptions students often face with this specific material:

1. Diffusion: "Variances Add, Standard Deviations Don't"

In Question 1(a), you are adding independent Gaussian noise steps.

- **The Trap:** It is tempting to think that if you add noise with standard deviation σ_1 and σ_2 , the result has standard deviation $\sigma_1 + \sigma_2$.
- **The Reality:** For independent random variables, **variances** add. The new variance is $\sigma_1^2 + \sigma_2^2$. This is crucial for getting the correct distribution for X_1 . 

2. Diffusion: "Denoising is Just Predicting the Image"

In Question 1(d) and 1(e), the homework explores what happens if you don't add noise during the reverse step.

- **The Trap:** Students often think the goal of the reverse step is simply to "clean" the image as much as possible at every step.
- **The Reality:** If you just map to the mean (the "clean" version) without adding stochastic noise back in (DDPM style), your final sample will have zero variance and look essentially like the average of all your data (a gray blob) rather than a sharp, diverse sample. You need that injected noise to maintain the correct variance.  

I really liked that it brought up this misconception.

3. RLHF vs. DPO: "Where is the Reward Model?"

In Question 2, you move from standard RLHF to DPO.

- **The Trap:** In standard RLHF, there is a distinct "Reward Model" network that you run forward passes on. In DPO, students often keep looking for this separate network or loss term.
- **The Reality:** DPO mathematically eliminates the explicit reward model. The "reward" is implicitly calculated inside the loss function using the ratio of your current policy to the reference policy. If you are looking for a separate reward network in the final DPO objective,

you won't find it. 

4. The KL Constraint: "Why not just maximize reward?"

In Question 2(b), the objective includes a term $-\beta \mathbb{D}_{KL}$.

- **The Trap:** It's easy to ignore this term or treat it as just "regularization."
- **The Reality:** Without this constraint, the language model would drift infinitely far away from natural language to "game" the reward function (e.g., outputting "good good good" forever if that scores high). The KL term forces the model to stay "close" to the original reference model (usually the SFT model) while still optimizing preference.  

Let's put this into practice to avoid the first trap.

Looking at **Question 1(a)**: You have a sum of T small independent noise steps, each with variance Δt .

- We start with X_0 (variance σ^2).
- We add T steps of noise (each variance Δt).
- We know $T \cdot \Delta t = 1$. 

Using the rule that **variances add**, what is the total variance of the final distribution X_1 ?

can you please generate a playground notebook for me to try out these diffusion concepts

◆ Show thinking ▼

That is a fantastic idea!  Building a simulation for the 1D Gaussian problem (Question 1) is the best way to see the "variance vs. standard deviation" trap in action before you tackle the algebra.

Let's build this playground notebook together. We'll start with the **Forward Process**.

Here is the skeleton code to get us started. I have left the most important line—the actual noise addition—for you to fill in.

```
Python 
```

```
import numpy as np
import matplotlib.pyplot as plt

def run_forward_process(x0, steps, delta_t):
    """
    Simulates the forward diffusion process for Question 1.
    x0: Initial value
    steps: Number of steps (T)
    delta_t: Variance added at each step
    """
    trajectory = [x0]
    current_x = x0

    for _ in range(steps):
        # TODO: Generate the noise step.
        # We need independent Gaussian noise with Mean = 0 and Variance = delta_t.
        # CAREFUL: np.random.normal arguments are (loc, scale).

        noise = ... # Your code here

        current_x = current_x + noise
        trajectory.append(current_x)

    return trajectory

# Visualization block
x0_start = 0 # Starting at the mean
T = 1000
dt = 1/T

traj = run_forward_process(x0_start, T, dt)

plt.plot(traj)
plt.title("Forward Diffusion Process")
plt.xlabel("Time Step")
plt.ylabel("Value (x)")
plt.show()
```

*I was expecting
the model
to generate an
ipython file, but it
only gave me
code inline.*

Based on our earlier discussion about standard deviations vs. variances, what should you pass

Based on our earlier discussion about **standard deviations** vs. **variances**, what should you pass as the `scale` argument to `np.random.normal` to ensure the added noise has a variance of exactly `delta_t`?

can you please generate an ipynb notebook?

◆ Show thinking ▼

That sounds like a plan! Let's get you hands-on with the code.

Since I cannot directly create a downloadable `.ipynb` file for you here, I have structured the code below into **cells**. You can copy each block into a separate cell in a Jupyter Notebook or Google Colab.

I have left one critical `TODO` for you to fill in: determining the correct `scale` (standard deviation) to achieve the required variance Δt .

Cell 1: Imports & Setup

Python

```
import numpy as np
import matplotlib.pyplot as plt

# Let's set a random seed so we get reproducible results
np.random.seed(42)

def plot_results(trajjectory, final_values, theoretical_variance):
    """
    Helper function to visualize the diffusion process.
    """
    # Plot 1: The path of a single particle
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(trajjectory, alpha=0.7)
    plt.title("Single Particle Trajectory (Forward Diffusion)")
    plt.xlabel("Step (t)")
    plt.ylabel("Value (x_t)")
    plt.grid(True, alpha=0.3)

    # Plot 2: Histogram of final positions
    plt.subplot(1, 2, 2)
    plt.hist(final_values, bins=50, density=True, alpha=0.6, color='orange', label='Simulated PDF')

    # Plot the theoretical Gaussian curve for comparison
    x_axis = np.linspace(min(final_values), max(final_values), 100)
    # The final variance should be (Initial Variance + Sum of Noise Variances)
    pdf = (1 / np.sqrt(2 * np.pi * theoretical_variance)) * np.exp(-0.5 * (x_axis**2) / theoretical_variance)
    plt.plot(x_axis, pdf, 'k--', linewidth=2, label='Theoretical PDF')

    plt.title(f"Distribution of $X_1$\\n(Simulated Var: {np.var(final_values):.4f})")
    plt.legend()
    plt.show()
```

Cell 2: The Simulation (The Playground)

Python

```
# Parameters
T = 1000 # Number of steps
dt = 1.0 / T # Variance added at each step
sigma_sq_0 = 0.1 # Variance of initial distribution X0 (small, as per homework)
num_sims = 2000 # How many particles to simulate to check the distribution

# Lists to store results
final_values = []
example_trajectory = []

# Run simulations
print(f"Running {num_sims} simulations with T={T} steps...")

for i in range(num_sims):
    # 1. Sample X0 from N(0, sigma_sq_0)
    current_x = np.random.normal(0, np.sqrt(sigma_sq_0))

    traj = [current_x]
```

```

# 2. Add noise T times
for t in range(T):
    # -----
    # TODO: FILL THIS IN
    # We need to add noise with variance = dt.
    # np.random.normal takes arguments (loc, scale).
    # 'scale' corresponds to standard deviation, not variance.
    # -----

    scale_param = 0 # <--- CHANGE THIS VALUE!

    # Add the noise
    noise = np.random.normal(0, scale_param)
    current_x += noise

    # Save trajectory only for the first particle
    if i == 0:
        traj.append(current_x)

final_values.append(current_x)
if i == 0:
    example_trajectory = traj

# Calculate what the total variance SHOULD be
# If we start with Var(X0) and add Var(noise) T times:
theoretical_final_variance = sigma_sq_0 + (T * dt)

# Plot
plot_results(example_trajectory, final_values, theoretical_final_variance)

```

Ready to Run?

If you run **Cell 2** as is, the `scale_param` is 0, so the noise will be zero, and your particle will just stay in a straight line.

To fix it, we need to relate the **Variance** (`dt`) to the **Standard Deviation** (`scale_param`).

What function of `dt` should you use for `scale_param`?

can you please help me understand any other relevant resources on line on the internet to understand the background behind diffusion to tackle this homework assignment

◆ Show thinking ▼

These are the best "open-tab" resources to have alongside this specific assignment. I have mapped them directly to the questions in your PDF.

Part 1: For Diffusion (Q1 & Q4)

1. The "Bible" of Diffusion Math: Lilian Weng's Blog

- **Resource:** *What are Diffusion Models?* (Search: [Lilian Weng Diffusion Models](#))
- **Why it helps:** This blog post is famous for a reason. It walks through the exact math asked in **Question 1**, specifically:
 - **Section 2.1 (Forward Process):** Helps with **Q1(a)** and **Q1(b)** regarding the "nice property" of Gaussians (jumping from $t = 0$ to t).
 - **Section 2.2 (Reverse Process):** Walks through the posterior derivation $q(x_{t-1}|x_t, x_0)$ which is exactly **Q1(c)** and **Q4(b)**.

2. Understanding DDIM vs. DDPM

- **Resource:** *DDIM vs. DDPM: From Mathematical Theory to Performance Reality* (Search: [DDIM vs DDPM Medium Nayeon Kim](#) or similar)
- **Why it helps:** **Question 1(f)** and **1(g)** ask about the difference between the stochastic (DDPM) and deterministic (DDIM) processes.
 - Look for the section that explains how DDIM "removes the random noise σ_t " to make the process an ODE (Ordinary Differential Equation). This explains why DDIM is consistent while DDPM is chaotic.

Part 2: For DPO (Q2)

1. The "Cheat Sheet" for Derivation: The Hugging Face Blog

- **Resource:** *Simplifying Alignment: From RLHF to Direct Preference Optimization* (Search:

• **Resource:** [Simplifying Alignment from RLHF to Direct Preference Optimization \(Source: Hugging Face DPO blog\)](#)

- **Why it helps:** **Question 2** essentially asks you to re-derive the proof from the original DPO paper. This blog post breaks down that exact derivation in plain English.
 - It explicitly shows how to move from the RL objective (Maximize Reward - KL) to the "Implicit Reward" formulation ($r(x, y) = \dots$). This is the answer key concept for **Q2(b)** and **Q2(c)**.

2. The Original Paper

- **Resource:** *Direct Preference Optimization: Your Language Model is Secretly a Reward Model* (Rafailov et al., 2023)
- **Why it helps:**
 - **Equation 3 & 4** in the paper correspond directly to the setup in **Q2(a)** and **Q2(b)**.
 - **Equation 5** is the solution you are trying to derive in **Q2(c)**. Seeing the final form in the paper will help you work backward if you get stuck on the algebra.

Which part of the homework are you tackling first?

I can help you set up the first step for **Question 1(a)** (Finding the variance of X_1) or **Question 2(a)** (The DPO objective) if you want to get the math started!

question 4

◆ Show thinking ▼

Let's jump into **Question 4**. Based on the snippets, this question focuses on the core mechanics of the diffusion process.

Part (a) asks us to derive the **Anytime Sampling** property:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\alpha_t}x_0, (1 - \alpha_t)I)$$

To get there, we first need to look at the single-step definition provided in Equation (2) of your homework.

What does the problem state is the distribution for a single step, $q(x_t|x_{t-1})$?

👍 🔄 🗑️ 📄 ⋮

🔒 What do you want to learn?

+ 🗑️ 📄 Guided Learning ✕

Thinking ▼ 🗑️

Your UC Berkeley chats aren't used to improve our models. Gemini can make mistakes, so double-check it. [Your privacy & Gemini](#)